JAVASCRIPT ESSENTIALS

Concurrency

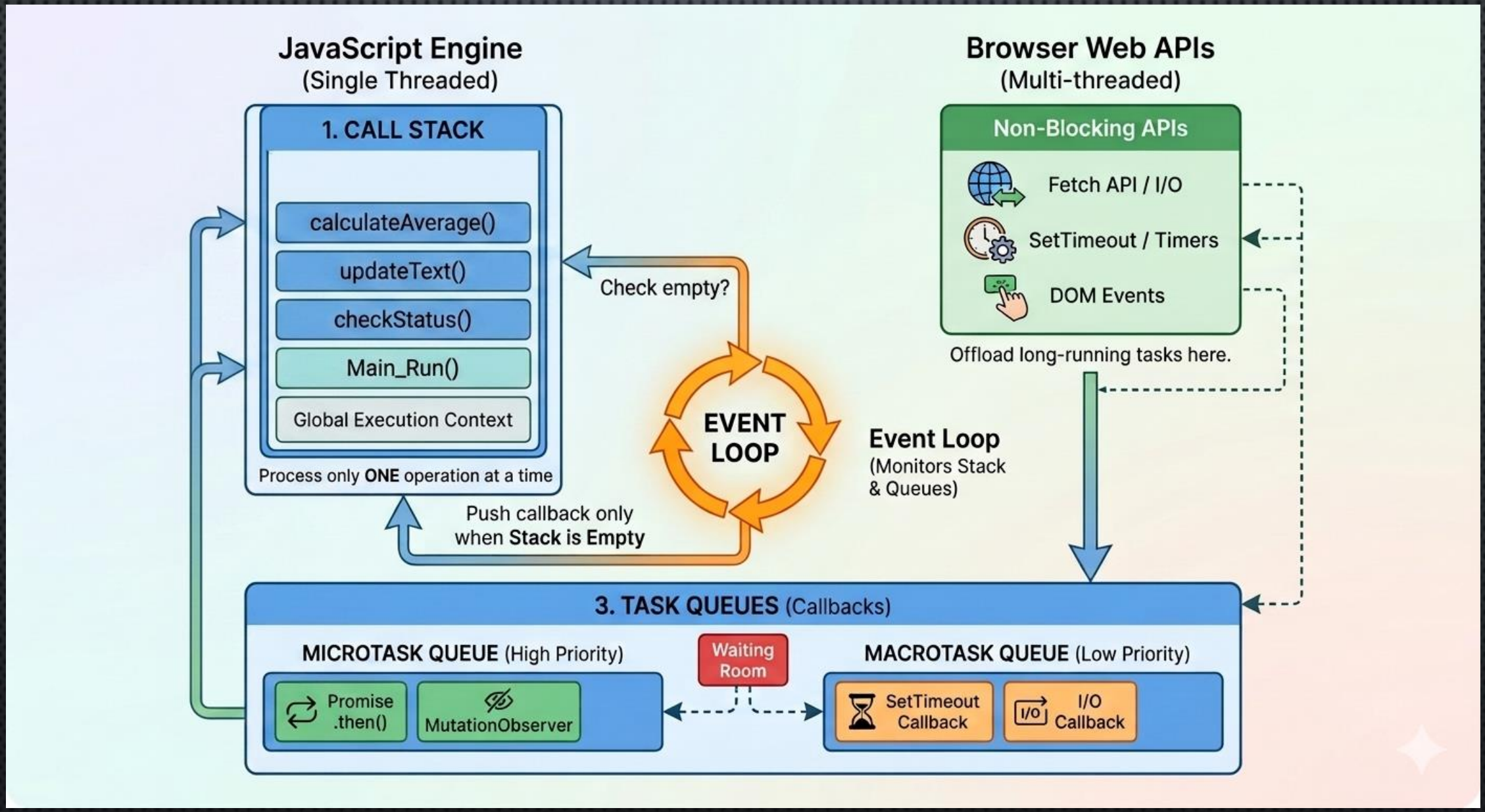
- Callbacks
  - Callbacks are functions passed as arguments to be executed later, typically when an asynchronous operation completes.
- Promises
  - A Promise represents a value that may be available now, in the future, or never. It has three states: pending, fulfilled, and rejected
- Async/await
  - The `async` keyword marks a function as asynchronous. It always returns a Promise. The `await` keyword pauses execution until a Promise settles.

EVENTLOOP

- Call Stack: This is where JavaScript keeps track of what is currently executing. When you call a function, it is pushed onto the Stack. When the function finishes, it is popped off.
- Web APIs / Node.js Environment: These are tools provided by the browser (like setTimeout, fetch, or DOM events) or Node.js. JavaScript delegates heavy work or waiting periods to them so it doesn't get blocked.
- Task Queue (or Macrotask Queue): This is where Web APIs send functions that are ready to be executed after their wait time is over (for example, the callback of a setTimeout).
- Microtask Queue: A special high-priority queue. This is primarily where callbacks from Promises (.then, .catch, .finally), queueMicrotask, or MutationObserver go.

Event Loop

- The Event Loop is an infinite loop whose sole job is to monitor the Call Stack and check if it is empty.
- If the Call Stack is empty, the Event Loop follows a very strict order of priority to clear out the queues:


JavaScript Engine
(Single Threaded)
1. CALL STACK
calculateAverage()
updateText()
checkStatus()
Main_Run()
Global Execution Context
Process only ONE operation at a time
Check empty?
EVENT LOOP
Push callback only when Stack is Empty
Event Loop
(Monitors Stack & Queues)
Browser Web APIs
(Multi-threaded)
Non-Blocking APIs
Fetch API / I/O
SetTimeout / Timers
DOM Events
Offload long-running tasks here.
3. TASK QUEUES (Callbacks)
MICROTASK QUEUE (High Priority)
Promise .then()
MutationObserver
Waiting Room
MACROTASK QUEUE (Low Priority)
SetTimeout Callback
I/O Callback

Errors

- By default, JavaScript provides generic error types like
  - Error
    - The baseline Error object is the parent of all other errors in JavaScript. It represents a generic runtime error
  - TypeError
    - A TypeError occurs when an operation is performed on a value that is of the wrong data type
  - ReferenceError
    - ReferenceError occurs when your code attempts to access or use a variable that does not exist, hasn't been initialized, or is completely out of scope.
- Custom Errors
  - User-defined class that extends the native JavaScript Error class

Error propagation

- Error propagation (often called "bubbling") is the mechanism where an error thrown inside a nested function automatically travels upwards through the call stack until it encounters a try...catch block. If no function in the call stack catches the error, it becomes an "uncaught exception," which can crash your application.

FETCH API

- Introduced in ES6, fetch() is the native, promise-based way to make asynchronous network requests in modern browsers. without needing to install external packages.
  - `fetch()` returns a Promise that resolves to a Response object.
  - `.json()`, `.text()`, `.blob()` on the response to parse the body.

Modules

- Is a way to split your code into separate, reusable files.
- Modules allow you to isolate variables, functions, and classes so they don't leak into the global scope, helping you maintain a clean and manageable codebase.
  - Default exports
  - Name exports

TESTS TYPES

- Unit Test, tests a **single unit of code** in isolation (usually a function).
  - No external dependencies
  - Fast
  - Easy to debug
- Integration Test, Tests how **multiple units work together**, Interactions between modules
  - Service layers
  - Module communication
  - API + logic combinations
- End-to-End (E2E) Test, **Tests the entire application** flow like a **real user** would.
  - Critical user flows
  - Login, checkout, navigation
  - Full system validation

Vitest

Vitest is a Vite-native testing framework that allows you to write, run, and manage tests (unit, integration, etc.) efficiently using a syntax very similar to Jest.

- Jest Compatible: It uses the same API (describe, it, expect, vi.fn()) as Jest, allowing developers to switch to Vitest without rewriting their existing test suites.
- Native ESM Support: It treats modern JavaScript modules (ESM) as first-class citizens, handling imports and exports natively without needing slow, complex compilation steps.
- Blazing Fast: It utilizes worker threads to run tests in parallel, leverages smart caching, and features an ultra-responsive "watch mode" that instantly reruns only the tests affected by your code changes.

VITEST

- Installing
  - Runs over node
  - Needs to be on a node project

```
mkdir mi-proyecto-js
cd mi-proyecto-js
npm init –y
npm install -D vitest     /  npm install vitest --save-dev
…
npm run test
```

MAIN FUNCTIONS

Function	Purpose
`describe`	Group tests
`test`	Define a test
`expect`	Assertions

Matchers

- ToBe, ToEqual, ToStrictEqual
- ToBeUndefined, ToBeDefined, toBeFalsy, toBeTruthy
- ToBeGreaterThan, toBeGreaterThanOrEqual, toBeLessThan, toBeLessThanOrEqual, toBeCloseTo, toMatch
- ToContain, ToMacthObject, ToHaveProperty, toHaveLength
- toThrow

Mocks

Mock is a simulated object or function that mimics the behavior of a real component in a controlled way.

- Isolation: Ensures a test failure means there is a bug in your code, not because a third-party server is down.
- Speed: Replaces slow network requests or database queries with near-instant internal memory swaps.
- Predictability: Allows you to easily simulate edge cases that are hard to trigger organically (like a 500 Internal Server Error or a network timeout).

EXERCICIO 06

- Se requiere escribir una applicacion para buscar la mejor fecha de viaje a argentina, se define una buena fecha aquella que en lo posible no tenga feriados de algun tipo.
- Entradas: un conjunto de fechas tentativas de inicio del viaje ['2026/06/10', '2026/06/15', '2026/06/20'] y duracion del viaje en dias
- Feriados malos: 'inamovible', 'puente'
- Feriados aceptables: 'trasladable'
- Salida: fechas ordenadas poniendo primero la mejor opción y luego es resto
- Incluir tests

Usar API
https://api.argentinadatos.com/v1/feriados/2026